KUBERNETES: orchestration de conteneurs

Kubernetes: orchestration de conteneurs

Origine de Kubernetes

Développement par Google : Kubernetes, souvent abrégé en K8s, est né dans Google et a été influencé par leur système interne appelé Borg.

Open-sourced en 2014 : Google a rendu Kubernetes open-source pour permettre à la communauté de contribuer et de le développer plus largement.

Kubernetes

Développement et Évolution

- **Croissance rapide** : Après son lancement, Kubernetes a rapidement gagné en popularité et est devenu un standard de fait pour la gestion de conteneurs à grande échelle.
- **Contribution de la Cloud Native Computing Foundation (CNCF)** : La CNCF, créée pour promouvoir les technologies cloud natives, a joué un rôle clé dans le développement et la promotion de Kubernetes.

Importance de Kubernetes dans la Conteneurisation moderne:

Standard de l'industrie : Kubernetes est devenu le standard de facto pour l'orchestration de conteneurs à grande échelle, largement adopté dans les environnements cloud et on-premise, devant des solutions comme Docker Swarm.:

Facilitation de la gestion des conteneurs : Kubernetes simplifie le déploiement, la gestion et la mise à l'échelle des applications conteneurisées grâce à des mécanismes natifs comme l'auto-scaling, l'auto-réparation et la gestion déclarative de l'état des applications.

Écosystème riche : Kubernetes bénéficie d'un écosystème riche et mature, comprenant de nombreux outils complémentaires (Helm, Prometheus, Istio, ArgoCD, etc.), permettant d'optimiser le déploiement, l'observabilité, la sécurité et l'automatisation des applications.

Adoption généralisée : Kubernetes est massivement adopté par les startups comme par les grandes entreprises, et constitue la base des principales plateformes cloud (GKE, EKS, AKS), confirmant son rôle central dans les architectures modernes.

Pourquoi Kubernetes est indispensable au-delà de Docker ?

- Docker gère **un conteneur ou une application**
- Kubernetes gère **un système distribué**
- Haute disponibilité
- Mise à l’échelle automatique
- Résilience aux pannes
- Déploiements continus sans interruption

Concepts de base de KUBERNETES

- **1. Nœuds (Nodes)**

  Un nœud est une machine physique ou virtuelle sur laquelle Kubernetes exécute des Pods, lesquels encapsulent un ou plusieurs conteneurs.

  - **Types de Nœuds :**
    - **Nœud du Control Plane (anciennement appelé Master Node)** : Gère le cluster et maintient l'état souhaité. (planification, décisions, API, configuration).
    - **Nœud de Travail (Worker Node) :** exécute les applications via des Pods.

Concepts de base de KUBERNETES

- **1. Nœuds (Nodes)**

  Un nœud est une machine physique ou virtuelle sur laquelle Kubernetes exécute des Pods, lesquels encapsulent un ou plusieurs conteneurs.

  - **Composants clés :**
    - **kubelet** : Agent présent sur chaque nœud de travail, chargé de s'assurer que les Pods définis par le Control Plane sont bien exécutés sur le nœud.
    - **kube-proxy** : Gère le routage réseau et l'exposition des Services Kubernetes en appliquant des règles réseau (iptables ou IPVS).
    - **Container Runtime** : Logiciel responsable de l'exécution des conteneurs (ex. containerd, CRI-O).

Concepts de base de KUBERNETES

- **2. Pods**

  - Plus petite unité déployable dans Kubernetes.
  - Un Pod encapsule un ou plusieurs conteneurs partageant le même contexte d'exécution.
  - **Caractéristiques:**
    - Les conteneurs d'un même Pod partagent :
      - le réseau (IP, ports),
      - le stockage (volumes),
      - et peuvent communiquer via localhost.
    - Chaque Pod possède une adresse IP unique, potentiellement temporaire.
    - Les Pods sont éphémères et peuvent être recréés automatiquement.
  - **Utilisation:**
    - Un Pod représente une instance d'exécution d'une application.
    - En pratique, les Pods sont créés et gérés par des contrôleurs (Deployment, StatefulSet, Job).

Concepts de base de KUBERNETES

- **2. Pods**
- **Pod mono-conteneur (le cas le plus courant)**
  - Une application = un conteneur
  - Facile à maintenir et à scaler
- **Pod multi-conteneurs (cas avancé)**
  - Pattern sidecar (ex : logger, proxy, agent)
  - Conteneurs fortement couplés
  - Toujours déployés ensemble

Concepts de base de KUBERNETES

- **3. Services**
- Un Service Kubernetes est une abstraction qui fournit un point d'accès stable (IP et DNS) à un ensemble dynamique de Pods.

  - **Types de Services :**
    - **ClusterIP :** accès interne au cluster (par défaut).
    - **NodePort :** exposition via un port statique sur chaque nœud.
    - **LoadBalancer :** exposition externe via un load balancer cloud.
    - **ExternalName :** redirection vers un service externe via DNS.

  - **Fonctionnement :**
    - Les Services utilisent des labels et selectors pour cibler les Pods.
    - L'adresse IP et le DNS du Service restent stables, même si les Pods changent.

Concepts de base de KUBERNETES

- **3. Services**

  Pourquoi ne pas accéder directement aux Pods ?

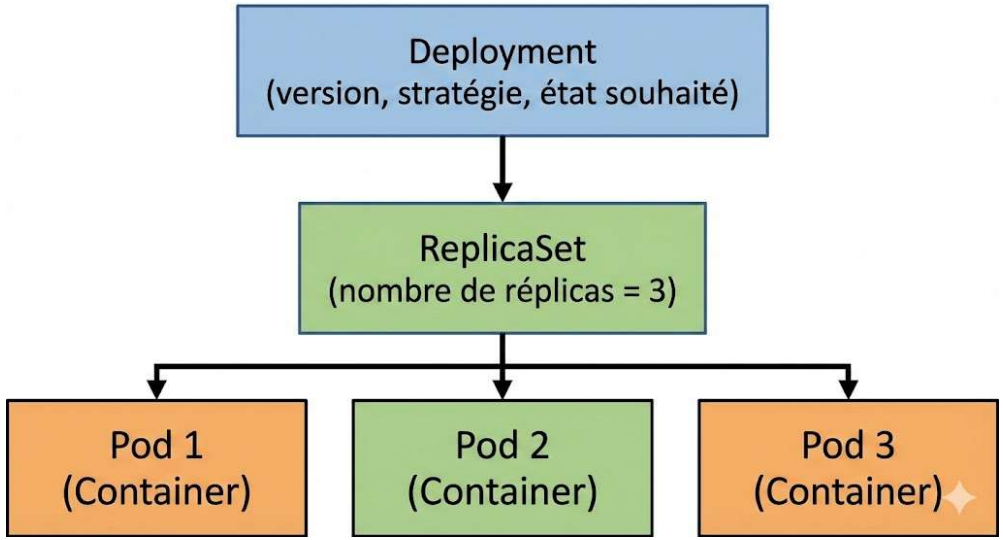
  - **Problèmes :**
    - IP des Pods changeante
    - Pods recréés automatiquement
    - Scalabilité dynamique
  - **Solution :**
    - Le Service agit comme un proxy logique et un load balancer interne

Concepts de base de KUBERNETES

- **4. Déploiements (Deployments)**
- Un Deployment permet de déclarer l'état souhaité d'une application et d'en assurer la disponibilité via des ReplicaSets.
  - **Caractéristiques:**
    - ➢ Gère la mise à jour des Pods de manière déclarative (rolling update).
    - ➢ Permet la mise à jour et le rollback des applications.
    - ➢ Gère le nombre de répliques et la recréation automatique des Pods en cas de panne.
  - **Processus:**
    - ➢ Défini via un fichier YAML précisant :
      - le nombre de répliques,
      - l'image du conteneur,
      - les labels et selectors,
      - la stratégie de déploiement.
    - ➢ Kubernetes maintient en permanence l'état réel conforme à l'état souhaité.

Concepts de base de KUBERNETES

- **4. Deployment vs Pod vs ReplicaSet**
- **Pod** : unité d'exécution
- **ReplicaSet** : garantit un nombre de Pods
- **Deployment** : gère les versions, les mises à jour et les ReplicaSets



Kubernetes Vs Docker Swarm

- Axes de comparaison:
  - Architecture et design
  - Scalabilité et gestion des ressources
  - Écosystème et intégrations
  - Cas d’usage et contextes d’utilisation

KUBERNETES Vs Docker SWARM

- **Design**

- **Kubernetes :**
  - Architecture modulaire et déclarative
  - Séparation claire entre contrôle (Control Plane) et exécution
  - Grande flexibilité mais complexité plus élevée.

- **Docker Swarm :**
  - Architecture simple et intégrée
  - Utilisation directe de l'API Docker
  - Mise en œuvre rapide, peu de composants

KUBERNETES Vs Docker SWARM

- **Scalabilité**

- **Kubernetes :**
  - Scalabilité à grande échelle (clusters massifs)
  - Gestion fine des ressources (CPU, mémoire)
  - Autoscaling via composants dédiés (HPA, VPA)
- **Docker Swarm :**
  - Scalabilité simple et rapide
  - Scaling manuel principalement
  - Adapté aux environnements de taille petite à moyenne

KUBERNETES Vs Docker SWARM

- **Écosystème**

- **Kubernetes :**
  - Écosystème très large (Helm, Prometheus, Istio, ArgoCD)
  - Support massif des cloud providers (GKE, EKS, AKS)
  - Innovation continue portée par la CNCF
- **Docker Swarm :**
  - Écosystème plus restreint
  - Intégration native avec Docker CLI et Docker Compose
  - Peu d’outils spécialisés tiers

KUBERNETES Vs Docker SWARM

- **En résumé,**
- **Kubernetes** est souvent privilégié pour des déploiements complexes et à grande échelle avec des exigences de gestion détaillée des ressources et une grande scalabilité,
- tandis que **Docker Swarm** est apprécié pour sa simplicité, sa facilité d'utilisation et son intégration parfaite avec Docker, étant idéal pour les environnements de taille moyenne et les équipes moins expérimentées en matière de conteneurisation.

Architecture de Kubernetes

I. Composants du Control Plane

Le Control Plane est le cœur du cluster Kubernetes.

Il est responsable de la gestion de l'état global du cluster, de la planification des applications et de la réaction aux événements.

1. API Server (kube-apiserver)

Agit comme la porte d'entrée du cluster Kubernetes.

Toutes les opérations et communications entre les composants du cluster passent par l'API Server.

Valide et configure les données pour les objets API tels que les pods, services, déploiements.

Il applique les mécanismes d'authentification, d'autorisation et de validation.

2. etcd

etcd est une base de données clé-valeur distribuée servant de source de vérité du cluster Kubernetes. Contient uniquement des métadonnées, jamais les données applicatives.

Architecture de Kubernetes

I. Composants du Control Plane

3. Scheduler (kube-scheduler)

Responsable de l'assignation des pods aux nodes.

Prend en compte divers facteurs tels que les ressources disponibles, les contraintes de politique, les affinités, les tolérances, etc.

Décide sur quel node un pod devrait s'exécuter.

Le scheduler ne lance pas les Pods, il choisit uniquement où ils doivent être exécutés.

4. Controllers

Exécutent des tâches de gestion de routine dans le cluster.

Incluent le Node Controller (qui gère les nodes), le ReplicaSet Controller (qui gère la réplication des pods), le EndpointSlice Controller, et d'autres.

Répondent aux changements dans l'état du cluster en travaillant pour ramener l'état actuel de l'ensemble du système à l'état souhaité.

Architecture de Kubernetes

I. **Composants des Nodes**

II. Les nodes sont les machines (physiques ou virtuelles) qui exécutent vos applications et workloads Kubernetes.

1. Kubelet

Agent qui s'exécute sur chaque nœud.

Assure que les Pods décrits par le Control Plane sont en cours d'exécution sur le nœud.

Communique avec le Control Plane pour recevoir les commandes et travaux, et pour envoyer des informations sur l'état du node et des pods.

2. Kube-proxy

Gère le réseau sur les nodes, y compris la mise en place des règles de réseau et le routage des connexions entrantes.

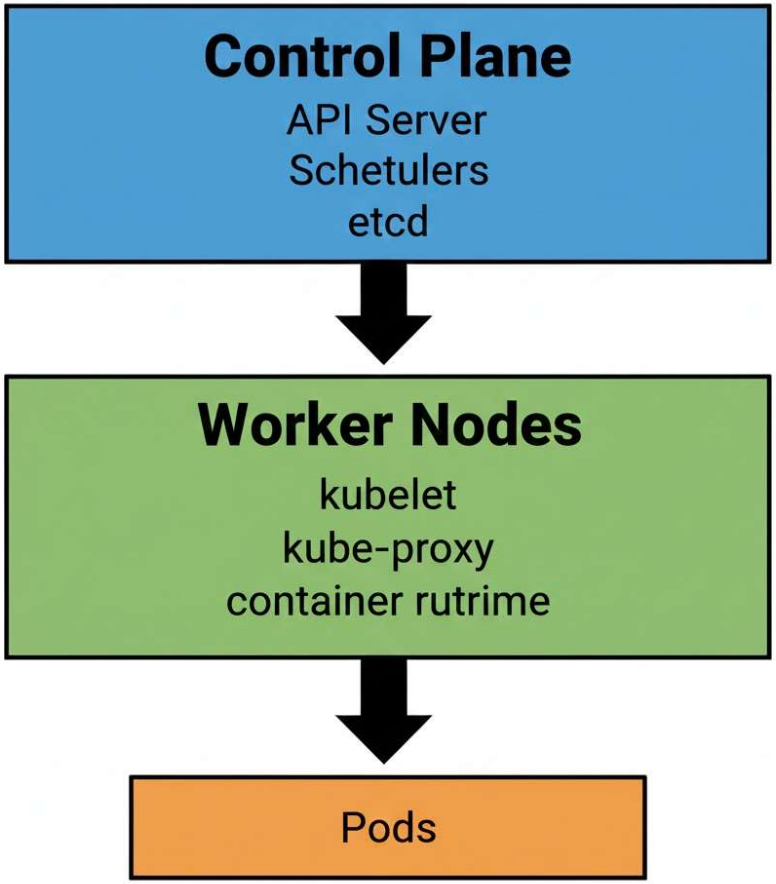
Permet la communication de réseau aux pods depuis les réseaux internes et externes au cluster.

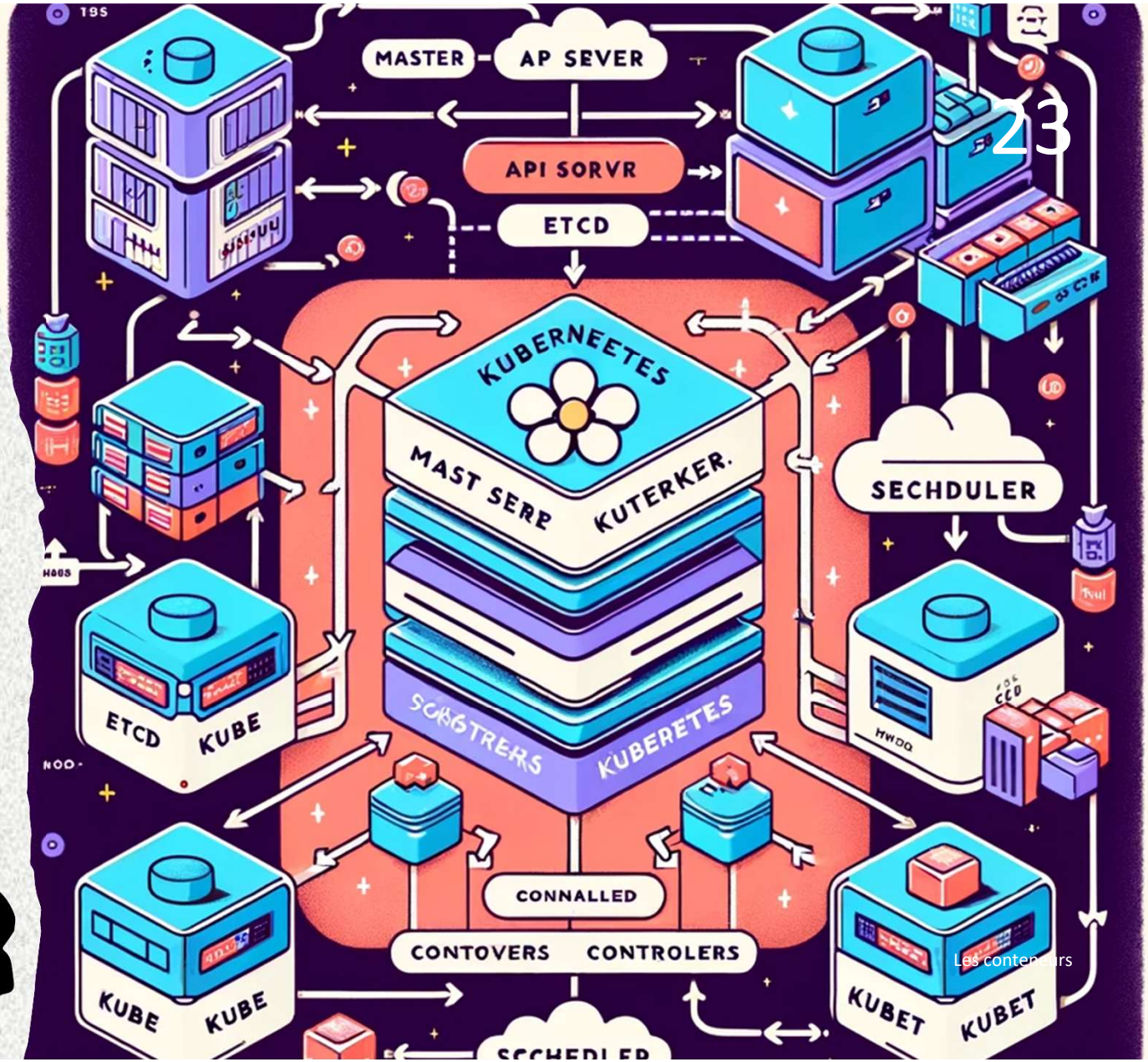
3. Container Runtime

Logiciel responsable de l'exécution des conteneurs.

Kubernetes est compatible avec plusieurs runtimes de conteneurs, tels que Docker, containerd, CRI-O.

Architecture de Kubernetes




MASTER
AP SEVER
API SORVR
ETCD
KUBERNEETES
MAST SERE
KUTERKER.
SECHDULER
ETCD
KUBE
SCHBTRERS
KUBERETES
CONNALLED
CONTOVERS
CONTROLERS
KUBE
KUBE
KUBET
KUBET

Installation et Configuration de Minikube

I. Fonctionnement Ensemble

- Le Master orchestre l'état souhaité du cluster, comme le déploiement d'applications ou la mise à l'échelle de services.

- Les Nodes exécutent les applications et workloads, en suivant les instructions du Master.

- Le Master communique avec les Nodes à travers l'API Server, et les Nodes utilisent le Kubelet pour exécuter les pods et les kube-proxys pour gérer le réseau.

MINIKUBE

Qu'est-ce que Minikube ?

- Un outil pour créer un cluster Kubernetes local.
- Simule un cluster Kubernetes dans une machine virtuelle ou directement sur l'ordinateur.
- Idéal pour l'apprentissage et le développement avec Kubernetes.

MINIKUBE

Utilisations Typiques de Minikube

- Pratiquer les commandes Kubernetes avec kubectl.
- Déployer et tester des applications en conteneurs.
- Expérimenter avec de nouvelles fonctionnalités Kubernetes.
- Développer et tester des microservices en local.

MINIKUBE

Utilisations Typiques de Minikube

- Pratiquer les commandes Kubernetes avec kubectl.
- Déployer et tester des applications en conteneurs.
- Expérimenter avec de nouvelles fonctionnalités Kubernetes.
- Développer et tester des microservices en local.

MINIKUBE

Installation de Minikube

Téléchargement de Minikube :

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

MINIKUBE

Catégorie	Commande	Description
Informations Générales	`kubectl version`	Affiche la version de kubectl et du cluster.
	`kubectl get pods`	Liste tous les pods dans le namespace par défaut.
	`kubectl get svc`	Liste tous les services dans le namespace par défaut.
	`kubectl get deployments`	Liste tous les déploiements dans le namespace par défaut.
	`kubectl get nodes`	Liste tous les nœuds du cluster.
Création et Suppression	`kubectl create -f <fichier.yaml>`	Crée une ressource à partir d'un fichier YAML.
	`kubectl delete pod <nom-du-pod>`	Supprime un pod spécifique.
Détails et Diagnostics	`kubectl describe pod <nom-du-pod>`	Affiche des informations détaillées sur un pod spécifique.
	`kubectl logs <nom-du-pod>`	Affiche les logs d'un pod spécifique.
	`kubectl exec <nom-du-pod> -- <commande>`	Exécute une commande spécifique dans un pod.

MINIKUBE

Catégorie	Commande	Description
Gestion des Déploiements	`kubectl scale deployment <nom-du-déploiement> --replicas=<nombre>`	Modifie le nombre de réplicas d'un déploiement.
	`kubectl set image deployment/<nom-du-déploiement> <nom-du-conteneur>=<nouvelle-image>`	Met à jour l'image d'un conteneur dans un déploiement.
	`kubectl rollout history deployment/<nom-du-déploiement>`	Affiche l'historique des mises à jour d'un déploiement.
	`kubectl rollout undo deployment/<nom-du-déploiement>`	Annule la dernière mise à jour d'un déploiement.
Commandes Avancées	`kubectl apply -f <fichier.yaml>`	Applique une configuration à partir d'un fichier YAML.
	`kubectl port-forward <nom-du-pod> <port-local>:<port-dans-le-pod>`	Redirige un port local vers un port dans un pod.
	`kubectl label pods <nom-du-pod> <cle>=<valeur>`	Ajoute ou modifie un label sur un pod spécifique.
	`kubectl run -i --tty <nom-du-pod> --image=<image> -- sh`	Lance un pod temporaire pour le débogage avec un shell interactif.